

Software Requirements Specification (SRS)

E-Commerce Web Application

Version 1.0

1. Introduction

1.1 Purpose

This document provides a detailed description of the requirements for the E-Commerce Web Application developed as part of the Software Engineering Lab Course. It defines functional and non-functional requirements, system features, constraints, and user interactions.

1.2 Scope

The system is a web-based e-commerce platform that allows users to:

- Register and authenticate accounts
- Browse products
- Add products to a shopping cart
- Place orders
- View order history
- Manage products (admin only)

The platform enables buyers to purchase products and administrators to manage inventory.

1.3 Definitions

- **User:** Any registered customer of the platform.
 - **Admin:** User with elevated privileges to manage products.
 - **Product:** Item available for purchase.
 - **Cart:** Temporary storage of selected products.
 - **Order:** Confirmed purchase transaction.
-

2. Overall Description

2.1 Product Perspective

The system is a standalone web application built using:

- Backend: Django (Python)
- Frontend: HTML, CSS, JavaScript
- Database: SQLite (development) / PostgreSQL (optional production)

The application follows the MVC (Model-View-Controller) architectural pattern (Django MVT).

2.2 Product Functions

High-level system capabilities:

1. User authentication (register/login/logout)
2. Product listing and detail view
3. Shopping cart management
4. Checkout and order creation
5. Order history viewing
6. Admin product management

2.3 User Classes and Characteristics

1. Guest User

- Can browse products
- Cannot place orders
- Must register/login to purchase

2. Registered User

- Can browse products
- Add/remove items from cart

- Place orders
- View order history

3. Admin User

- All registered user permissions
- Add, update, delete products
- Manage inventory

2.4 Operating Environment

- Web browser (Chrome, Firefox, Edge, Safari)
- Server-side: Python 3.x
- Django Framework
- SQL database
- Hosted locally or on cloud platform

2.5 Design Constraints

- Must use Django framework
- Must follow CS50 Web project guidelines
- Use relational database
- Implement authentication securely

2.6 Assumptions and Dependencies

- Users have internet access
- Server environment supports Python and Django
- Database server is available

3. System Features

3.1 User Registration

Description

Allows new users to create accounts.

Functional Requirements

- FR1: User must provide username, email, and password.
- FR2: System must validate unique username.
- FR3: Password must be securely hashed.
- FR4: User must confirm password.

3.2 User Authentication

Description

Allows registered users to log in and log out.

Functional Requirements

- FR5: User must log in with valid credentials.
- FR6: Invalid login attempts must show error message.
- FR7: System must maintain user session.
- FR8: User must be able to log out securely.

3.3 Product Management (Admin)

Description

Admin users manage products.

Functional Requirements

- FR9: Admin can create new products.

- FR10: Admin can edit product details.
 - FR11: Admin can delete products.
 - FR12: Each product must have:
 - Name
 - Description
 - Price
 - Image
 - Stock quantity
-

3.4 Product Browsing

Description

Users can view available products.

Functional Requirements

- FR13: System must display all products.
 - FR14: Users can view product details.
 - FR15: System must show price and availability.
 - FR16: Products should be paginated if large in number.
-

3.5 Shopping Cart

Description

Users can manage cart items before checkout.

Functional Requirements

- FR17: User can add product to cart.
 - FR18: User can remove product from cart.
 - FR19: User can update quantity.
 - FR20: System must calculate total price dynamically.
 - FR21: Cart must persist during session.
-

3.6 Checkout & Orders

Description

Users can place orders for items in cart.

Functional Requirements

- FR22: User must confirm order before placement.
 - FR23: System must create order record.
 - FR24: System must reduce stock quantity.
 - FR25: Order must include:
 - Order ID
 - User
 - Items
 - Total price
 - Date
 - Status
-

3.7 Order History

Description

Users can view past orders.

Functional Requirements

- FR26: User can view list of past orders.

- FR27: User can view detailed order information.
- FR28: Orders must display status (Pending, Shipped, Delivered).

4. External Interface Requirements

4.1 User Interface

- Responsive design
- Navigation bar with:
 - Home
 - Cart
 - Orders
 - Login/Logout
- Clear product layout
- Error messages must be displayed clearly

4.2 Hardware Interface

No specific hardware requirements beyond a device capable of running a modern web browser.

4.3 Software Interface

- Django authentication system
- Relational database (SQLite/PostgreSQL)

5. Non-Functional Requirements

5.1 Performance Requirements

- Page load time ≤ 3 seconds under normal conditions
- System must support at least 100 concurrent users (basic scalability target)

5.2 Security Requirements

- Passwords must be hashed
- CSRF protection must be enabled
- Only admin users can access product management
- Session management must be secure

5.3 Usability Requirements

- Simple and intuitive interface
- Clear navigation
- Mobile-responsive layout

5.4 Reliability Requirements

- System uptime $\geq 95\%$ (development target)
- Database transactions must ensure data consistency

5.5 Maintainability

- Code must follow Django best practices
- Modular structure
- Proper documentation and comments

6. Database Requirements (Logical Model)

Entities:

User

- id (PK)

- username
- email
- password

Product

- id (PK)
- name
- description
- price
- stock
- image

Cart

- id (PK)
- user (FK)

CartItem

- id (PK)
- cart (FK)
- product (FK)
- quantity

Order

- id (PK)
- user (FK)
- total_price
- date
- status

OrderItem

- id (PK)
- order (FK)
- product (FK)
- quantity
- price

7. Future Enhancements

- Online payment integration
- Product reviews and ratings
- Search and filtering
- Wishlist functionality
- Email notifications
- Admin dashboard analytics